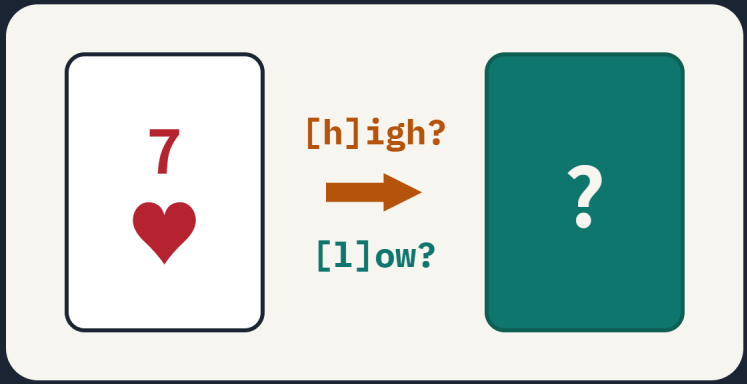
エンジニア研修 ・ オブジェクト指向 開発演習

ハイ＆ロー演習

まとめて書いてから、
責務を見つけて分ける



責務の分割　カプセル化（不変条件）　多態性

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

01

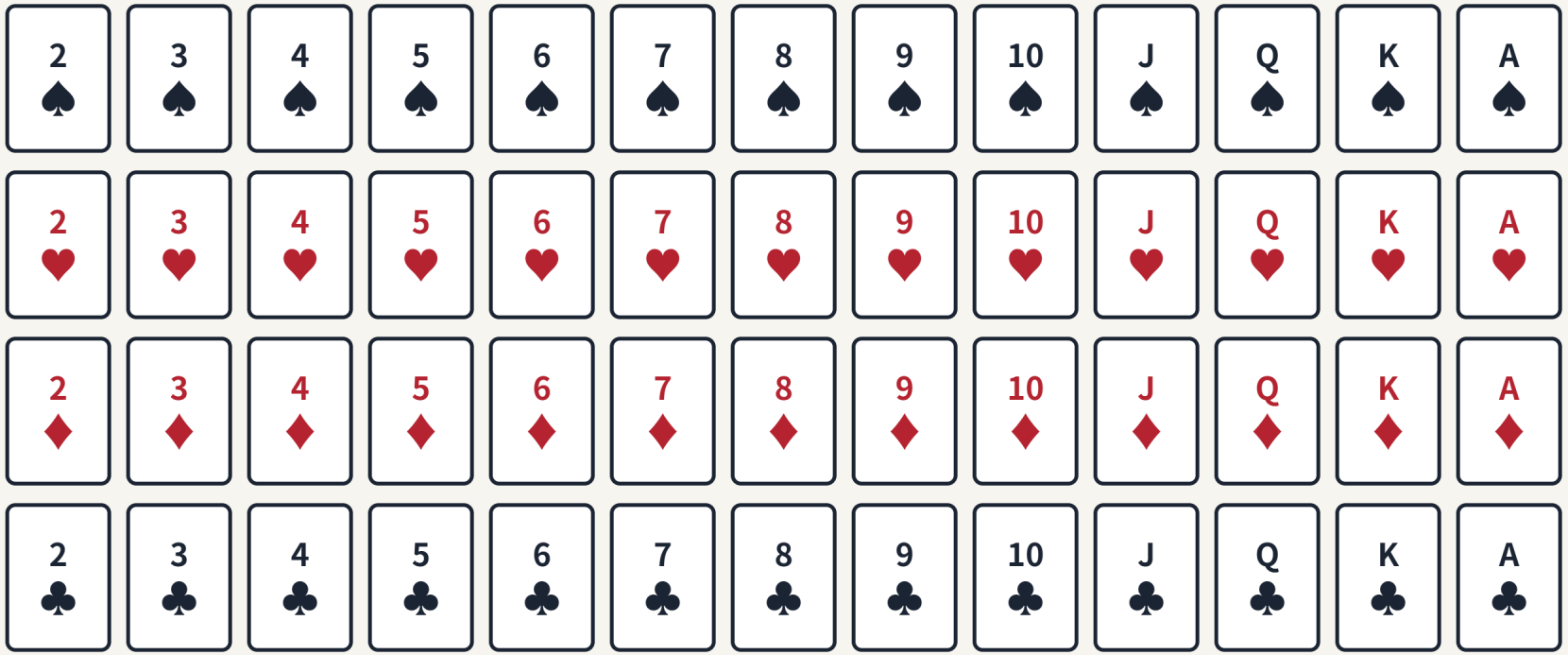
ルール

今回のハイ＆ローを知る

ルール

使うカードは52枚

トランプから**ジョーカーを除いた52枚**を使います。スペード・ハート・ダイヤ・クラブの4種類のマークに、2〜A の13枚ずつがあります。

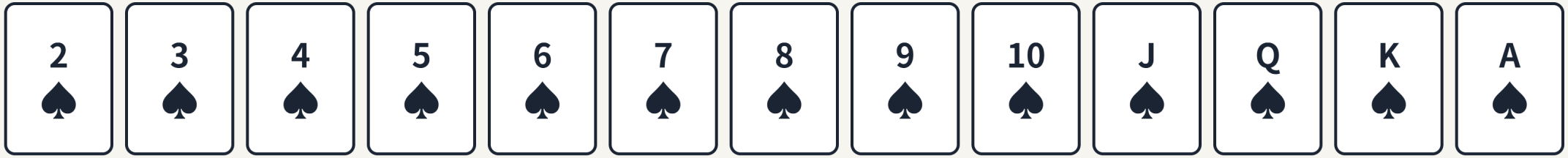


ルール

カードの強さ

カードの強さは、数字で決まります。A が一番強く、2 が一番弱くなります。

← 弱い　　強い →



A ＞ K ＞ Q ＞ J ＞ 10 ＞ 9 ＞ 8 ＞ 7 ＞ 6 ＞ 5 ＞ 4 ＞ 3 ＞ 2

ルール

ゲームの流れ

1

山札から1枚めくる

よく切った52枚の山札から、最初の1枚をめくります。

2

次のカードを予想する

次にめくるカードが、今のカードより強いか弱いかを予想します。

3

めくって確かめる

次のカードをめくり、予想が当たったかを確かめます。

予想が当たったら、連続正解数が1つ増え、めくったカードを今のカードにして 2 に戻ります。

予想が外れるか、山札がなくなったら、ゲームは終わりです。

何回連続で予想を当てられたか（連続正解数）を競います。

ルール

1人で遊ぶ

画面に今のカードが表示されます。次のカードのほうが強いと思えば **h**（High）、弱いと思えば **l**（Low）を入力します。

[h]igh

次のカードのほうが強い

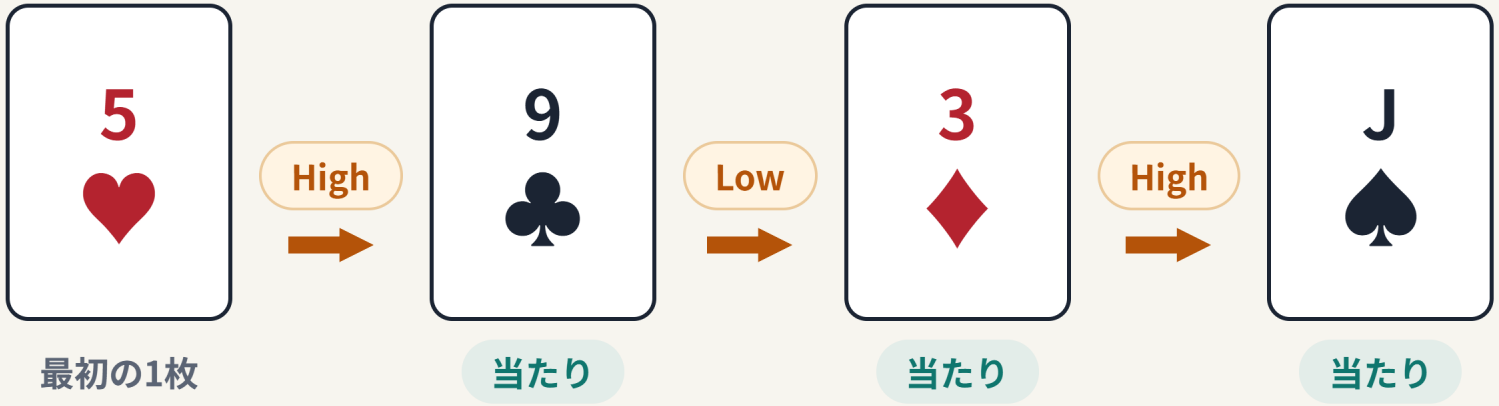
[l]ow

次のカードのほうが弱い

```
Current card: 7 of Hearts. Is the next card [h]igh or [l]ow? > h
```

ルール ・ 遊び方の例

例1：予想が当たり続ける



結果：3回続けて当たったので、連続正解数は3。まだ続く

ルール ・ 遊び方の例

例2：予想が外れる



結果：2回目の予想が外れたので、ここで終わり。連続正解数は1

ルール ・ 遊び方の例

例3：山札がなくなる



結果：52枚目をめくって山札がなくなったので、ここで終わり

02

進め方

まとめて書いてから、分ける

進め方

分け方を自分で決める

この演習では、作るクラスをあらかじめ決めません。まず動くものを作り、その流れを見ながら分け方を考えます。

1 はじめに、全体を動かす

すべての処理を main に書いて、ゲームを最後まで動かす

ゲームの流れ全体を、1か所で見渡せるようにする

2 流れを見ながら、分ける

どこで分けるかは、動くコードを見ながら自分たちで決める

分け方の正解は1つではない。なぜそう分けたかを説明できるようにする

先に全体を動かすと、何と何がつながっているかが見える。

進め方

4つの段階で分ける

一度に分けようとせず、小さな段階を順に進みます。どの段階でも、ゲームの動きは変えません。

1

main に書き下す

クラスもメソッドも作らず、すべてを main に書いて動かす

2

メソッドに切り出す

ひとまとまりの処理に、名前をつけて切り出す

3

クラスに切り出す

いつも一緒に使うデータと処理を、1つにまとめる

4

役割で分ける

種類ごとに違うところを、それぞれに任せる

名前をつける → まとめる → 任せる、の順に、少しずつ形を変えていく。

進め方

切り出す手がかり

コードの中に次のようなところを見つけたら、切り出すきっかけになります。

見つけたもの	切り出し方
同じ処理が、2か所以上に書かれている	メソッドにして、1か所にまとめる
コメントで「〜する」と説明しているかたまり	メソッドにする（コメントが名前の候補）
いつも一緒に受け渡ししているデータ	クラスにまとめる
同じデータを読み書きするメソッドの集まり	データと一緒に、クラスに移す
種類によって、if で書き分けている処理	種類ごとのクラスに任せる（多態性）

進め方

動きを変えずに分ける

切り出すたびに、**同じシードと同じ入力**で動かし、表示が変わらないことを確かめます。切り出す前の表示を before.txt に保存しておきます。

```
./mvnw -q compile
printf 'h\nl\nl\nh\n' | java -cp target/classes highlow.HighAndLowApp 111 > before.txt

# （切り出したあと）
./mvnw -q compile
printf 'h\nl\nl\nh\n' | java -cp target/classes highlow.HighAndLowApp 111 > after.txt
diff before.txt after.txt
```

ポイント	理由
引数でシード（111）を渡す	山札の切り方が、毎回同じになる
printf で入力を渡す	High・Low の予想が、毎回同じになる
diff で何も表示されない	切り出しの前後で、動きが変わっていない

※ シード（seed）：乱数（ランダムな値）を作るときの「元になる数」です。同じシードを渡すと、毎回同じ順番で同じ乱数が出てきます。

03

課題

書いて、分けて、確かめる

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

HighAndLowApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変えてみる

早く終わった人向け

演習 ・ 課題1

ルールの中にある法則を見つけよう

3〜4人で以下を話し合いましょう。

1. 「カード」が強い、とはどういう状態か
2. 「予想」が当たる、とはどういう状態か
3. 次に同じ強さのカードがめくられたとき、「予想」はどうなるか
4. 「ゲーム」が終わる、とはどういう状態か
5. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
   （それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

ヒント：3は、ルールのスライドには直接書かれていない場面です。2で考えた「当たり」の条件に当てはめて、どうなるかを考えてみましょう。5は、ルールの説明に出てくる名詞と動詞を拾い出すところから始めます。

検討例 ・ 課題1

強さ・当たり・Draw・終わり・もの を、言葉で言い表す

1.「カード」が強い、とはどういう状態か

→ ランク（数字）が大きいこと。スートは関係ない

2.「予想」が当たる、とはどういう状態か

→ High なら次のカードが強いこと、Low なら次のカードが弱いこと

3. 次に同じ強さのカードがめくられたとき、「予想」はどうなるか

→ 当たりでも外れでもない（Draw）。数えないが、ゲームは続く

4.「ゲーム」が終わる、とはどういう状態か

→ 予想が外れるか、山札がなくなること

5. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
（それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

→ カード（強さを比べる）、山札（1枚めくられる）、予想（当たりかを決める）、結果（数えるか・続けるか）

検討例 ・ 課題1

見つけた法則が、分け方の手がかりになる

対象	見つけた法則	分けるときに考えること
カード	強さ ＝ ランクの大きさ	ランクを強さの順に並べ、並び順で比べる
予想	当たり ＝ High なら強い、Low なら弱い	予想ごとに、当たりになる条件を持たせる
予想	Draw ＝ 同じ強さ	当たり・外れのほかに、3つ目の結果を用意する
結果	当たりだけ数え、外れだけで終わる	結果ごとに、加える点と続けられるかを持たせる
もの	カード・山札・予想・結果・プレイヤー	名詞はクラスの、動詞はメソッドの候補

いきなりクラスを決めなくてよい。まず main に書き、ここで見つけた「もの」を思い出しながら分けていく。

Maven Wrapper を取得する

Maven Wrapper を直接ダウンロードし、使う Maven のバージョンを指定します。

```
WRAPPER_VERSION=3.3.4
MAVEN_VERSION=3.10.0

mkdir highlow && cd highlow

curl -fsSLO "https://repo.maven.apache.org/maven2/org/apache/maven/wrapper/maven-wrapper-
distribution/${WRAPPER_VERSION}/maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip"
unzip -l maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
unzip maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
rm maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip

mkdir -p .mvn/wrapper
cat > .mvn/wrapper/maven-wrapper.properties <<EOF
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/${MAVEN_VERSION}/apache-
maven-${MAVEN_VERSION}-bin.zip
EOF

chmod +x mvnw
```

準備 ・ プロジェクトを用意する（2/3）

Maven Wrapper でひな形を作る

取得した mvnw で、Maven 公式のテンプレート（archetype）からプロジェクトを生成します。highlow フォルダの中で実行してください。

```
./mvnw archetype:generate \
  -DarchetypeGroupId=org.apache.maven.archetypes \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DjavaCompilerVersion=25 \
  -DgroupId=com.example \
  -DartifactId=highlow \
  -Dpackage=highlow \
  -DinteractiveMode=false

mv highlow/* ./ && rm -rf highlow/
```

archetype:generate は、artifactId と同じ名前（highlow）のフォルダを作って、その中にプロジェクトを生成します。最後の行で、生成された中身を highlow の直下に移しています。

-DjavaCompilerVersion=25 で、使う Java のバージョンを 25 に指定しています（指定しないと 17 になります）。あらかじめ JDK 25 をインストールしておいてください。

準備 ・ プロジェクトを用意する（3/3）

.gitignore を作る

ビルド結果や IDE の設定ファイルを、Git の管理から外します。highlow フォルダの中で実行してください。

```
cat > .gitignore <<'EOF'
target/
.idea/
*.iml
.vscode/
.DS_Store
mvnwDebug*
before.txt
after.txt
EOF
```

できあがる構成

```
highlow/
├── .gitignore
├── .mvn/wrapper/
│   └── maven-wrapper.properties
├── mvnw
├── mvnw.cmd
├── pom.xml
└── src/
    ├── main/java/highlow/
    └── test/java/highlow/
```

.mvn フォルダは Git の管理から外さないでください。使う Maven のバージョンが書かれています。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

HighAndLowApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変えてみる

早く終わった人向け

演習 ・ 課題2

main に書き下す

HighAndLowApp クラスの main メソッドの中に、ゲームのすべてを書いて動かします。まずは動けばよく、きれいに書く必要はありません。

1. 新しいクラスもメソッドも作らず、main 1つの中に書く
2. カードは int（0〜51 の番号）で表してよい
3. 実行時の引数でシードを受け取り、new Random(シード) と Collections.shuffle で山札を切る
4. 画面の表示は、次のページの表示例に合わせる

完成したら、課題3以降で比べるために before.txt を作っておく（進め方「動きを変えずに分ける」のコマンド）

画面の表示例

画面の表示はすべて英語にします。カードは「7 of Hearts」のように、ランクとスートで表示します。

```
$ java -cp target/classes highlow.HighAndLowApp 111
First card: 6 of Clubs
Current card: 6 of Clubs. Is the next card [h]igh or [l]ow? > h
  Next card: 8 of Clubs ... Hit! (streak: 1, 50 cards left)
Current card: 8 of Clubs. Is the next card [h]igh or [l]ow? > l
  Next card: 8 of Hearts ... Draw (streak: 1, 49 cards left)
Current card: 8 of Hearts. Is the next card [h]igh or [l]ow? > l
  Next card: 7 of Diamonds ... Hit! (streak: 2, 48 cards left)
Current card: 7 of Diamonds. Is the next card [h]igh or [l]ow? > h
  Next card: 3 of Spades ... Miss... (streak: 2, 47 cards left)

Game over. Your streak: 2
```

山札がなくなって終わったときは、Game over の前に「No cards left.」と表示します。

検討例 ・ 課題2

main の中身を、かたまりで見る

書き上げた main を上から読むと、次のかたまりが並んでいます。

かたまり	やっていること
山札を作る	0〜51 の番号を並べ、シードを使って切る
最初の1枚	1枚めくって表示する
予想を受け取る	h か l が入力されるまで聞き直す
めくって判定する	同じ強さなら Draw、当たりなら Hit!（数える）、外れなら Miss...
終わり	外れるか山札がなくなるまで繰り返し、連続正解数を表示する

番号 c のカードは、c % 13 でランク（0 が 2、12 が A）、c / 13 でスートがわかる。ランクの番号どうしを比べれば強さがわかる

解答例 ・ 課題2（1/2）

main に書き下した例

HighAndLowApp.java

```
public class HighAndLowApp {
    public static void main(String[] args) {
        long seed = args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
        Random random = new Random(seed);
        Scanner scanner = new Scanner(System.in);
        // ランクは弱い順：0 が 2、12 が A
        String[] ranks = {"2", "3", "4", "5", "6", "7", "8",
                "9", "10", "J", "Q", "K", "A"};
        String[] suits = {"Spades", "Hearts", "Diamonds", "Clubs"};

        // 山札：0〜51 の番号で 52 枚を表し、よく切る
        List<Integer> deck = new ArrayList<>();
        for (int i = 0; i < 52; i++) {
            deck.add(i);
        }
        Collections.shuffle(deck, random);

        int current = deck.remove(0);
        System.out.println("First card: " + ranks[current % 13]
                + " of " + suits[current / 13]);

        int streak = 0;
        boolean missed = false;
        while (!deck.isEmpty() && !missed) {
```

HighAndLowApp.java（続き）

```
            String currentLabel = ranks[current % 13] + " of "
                    + suits[current / 13];
            String input = "";
            while (!input.equals("h") && !input.equals("l")) {
                System.out.print("Current card: " + currentLabel
                        + ". Is the next card [h]igh or [l]ow? > ");
                input = scanner.nextLine().trim();
            }

            int next = deck.remove(0);
            String result;
            if (next % 13 == current % 13) {
                result = "Draw";
            } else if ((next % 13 > current % 13)
                    == input.equals("h")) {
                result = "Hit!";
                streak++;
            } else {
                result = "Miss...";
                missed = true;
            }
            System.out.println("  Next card: " + ranks[next % 13]
                    + " of " + suits[next / 13] + " ... " + result
                    + " (streak: " + streak + ", " + deck.size()
                    + " cards left)");
```

解答例 ・ 課題2（2/2）

main に書き下した例

HighAndLowApp.java（続き）

```
            current = next;
        }

        if (!missed) {
            System.out.println("No cards left.");
        }
        System.out.println();
        System.out.println("Game over. Your streak: " + streak);
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

HighAndLowApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変えてみる

早く終わった人向け

演習 ・ 課題3

メソッドに切り出す

課題2の main から、ひとまとまりの処理を static メソッドとして切り出します。

1. main を読み、ひとまとまりの処理に「〜する」と名前をつける
2. 同じ処理が2か所にあれば、1つのメソッドにまとめる
3. 1つ切り出すごとにコンパイルして動かし、diff before.txt after.txt で表示が変わらないことを確かめる

条件：main を読むだけで、ゲームの流れ（最初の1枚 → 予想 → めくって判定 → 終わりの表示）がわかるようにする

検討例 ・ 課題3

メソッドに切り出す

役割	作るもの
引数からシードを決める	seedFrom(String[] args): long
52 枚をそろえて切る	newShuffledDeck(Random random): List<Integer>
カードの表示名を作る（例：7 of Hearts）	label(int card): String
h か l を受け取る（High なら true）	askHigh(Scanner, int current): boolean
当たり・Draw・外れを判定する	judge(current, next, high): String
めくった結果を表示する	printResult(next, result, streak, left): void
終わりの表示をする	printGameOver(missed, streak): void

検討例 ・ 課題3 クラス図

同じデータを受け取るメソッドが集まっている

課題3のメソッドを、受け取るデータで分けてみます。いつも同じデータを受け取るメソッドは、そのデータと一緒にクラスにまとめる候補です。

受け取るデータ	メソッド	次の課題では
List<Integer> deck	newShuffledDeck()、remove(0)	山札（Deck）にまとめる
int card	label()、judge() の % 13	カード（Card）にまとめる
boolean high	askHigh()、judge()	課題5で予想（Guess）に
結果の文字列	judge()、main の if	課題5で結果（Result）に

「どのデータを受け取るか」が、クラスの境目を教えてくれる。

解答例 ・ 課題3（1/2）

メソッドに切り出した例

HighAndLowApp.java

```
public class HighAndLowApp {
    private static final String[] RANKS = {"2", "3", "4", "5",
            "6", "7", "8", "9", "10", "J", "Q", "K", "A"};
    private static final String[] SUITS =
            {"Spades", "Hearts", "Diamonds", "Clubs"};

    public static void main(String[] args) {
        Random random = new Random(seedFrom(args));
        Scanner scanner = new Scanner(System.in);

        List<Integer> deck = newShuffledDeck(random);
        int current = deck.remove(0);
        System.out.println("First card: " + label(current));

        int streak = 0;
        boolean missed = false;
        while (!deck.isEmpty() && !missed) {
            boolean high = askHigh(scanner, current);
            int next = deck.remove(0);
            String result = judge(current, next, high);
            if (result.equals("Hit!")) {
                streak++;
            } else if (result.equals("Miss...")) {
                missed = true;
            }
```

HighAndLowApp.java（続き）

```
            printResult(next, result, streak, deck.size());
            current = next;
        }
        printGameOver(missed, streak);
    }

    static long seedFrom(String[] args) {
        return args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
    }

    static List<Integer> newShuffledDeck(Random random) {
        List<Integer> deck = new ArrayList<>();
        for (int i = 0; i < 52; i++) {
            deck.add(i);
        }
        Collections.shuffle(deck, random);
        return deck;
    }

    static String label(int card) {
        return RANKS[card % 13] + " of " + SUITS[card / 13];
    }

    static boolean askHigh(Scanner scanner, int current) {
```

解答例 ・ 課題3（2/2）

メソッドに切り出した例

HighAndLowApp.java（続き）

```
        String input = "";
        while (!input.equals("h") && !input.equals("l")) {
            System.out.print("Current card: " + label(current)
                    + ". Is the next card [h]igh or [l]ow? > ");
            input = scanner.nextLine().trim();
        }
        return input.equals("h");
    }

    static String judge(int current, int next, boolean high) {
        if (next % 13 == current % 13) {
            return "Draw";
        }
        if ((next % 13 > current % 13) == high) {
            return "Hit!";
        }
        return "Miss...";
    }

    static void printResult(int next, String result, int streak,
            int left) {
        System.out.println("  Next card: " + label(next) + " ... "
                + result + " (streak: " + streak + ", " + left
                + " cards left)");
    }
```

HighAndLowApp.java（続き）

```
    static void printGameOver(boolean missed, int streak) {
        if (!missed) {
            System.out.println("No cards left.");
        }
        System.out.println();
        System.out.println("Game over. Your streak: " + streak);
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

HighAndLowApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変えてみる

早く終わった人向け

演習 ・ 課題4

クラスに切り出す

課題3で、いつも同じデータを受け取っていたメソッドを、そのデータと一緒にクラスへまとめます。int のカードも、ランクとスートを持つものに置き換えます。

1. フィールドはすべて private にし、山札の List を外から直接変更できないようにする
2. カードは、ランクとスートが必ずそろっている
3. ランクの強さを if や switch で書き分けず、enum の並び順で比べる
4. 山札から同じカードは2度出ず、空の山札からは引けない

ヒント：山札は課題3と同じ順（スートごとに 2, 3, …, A）に並べてから切ると、同じシードで同じ表示になる

検討例 ・ 課題4

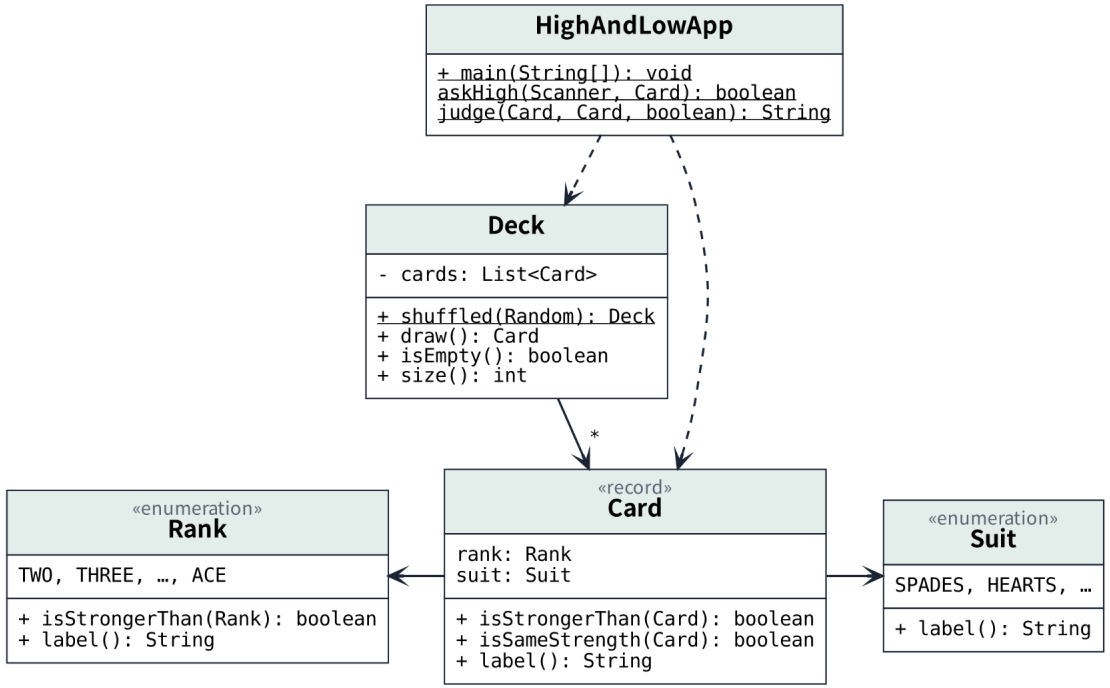
クラスに切り出す

役割	作るもの
13種類のランク（2〜A）を、弱い順に並べたもの	enum Rank：TWO, THREE, …, ACE
自分が相手のランクより強いかを検査	Rank：isStrongerThan(Rank): boolean
4種類のスート	enum Suit：label()
ランクとスートを組で持つ	record Card(Rank rank, Suit suit)
強いか・同じ強さかを検査、表示名を作る	Card：isStrongerThan(), isSameStrength(), label()
52 枚をそろえて切った山札を作る	Deck：static shuffled(Random): Deck
1枚引く・空かを検査・残り枚数	Deck：draw(), isEmpty(), size()

HighAndLowApp には、予想の受け取りと判定（judge）が残る

検討例 ・ 課題4 クラス図

強さの順は、Rank の並び順そのもの



«enumeration» Rank

13個の定数を、弱い順（TWO から ACE）に並べる

«record» Card

ランクとスートを組で持ち、強さを比べる

Deck

カードの List を private で持ち、draw() でしか減らせない

HighAndLowApp

予想の受け取りと判定（judge）が、まだ残っている

解答例 ・ 課題4（1/4）

クラスに切り出した例

Rank.java

```
/**
 * カードのランク。
 * 定数の並び順がそのまま強さの順（2 が最も弱く、A が最も強い）。
 */
public enum Rank {
    TWO("2"),
    THREE("3"),
    FOUR("4"),
    FIVE("5"),
    SIX("6"),
    SEVEN("7"),
    EIGHT("8"),
    NINE("9"),
    TEN("10"),
    JACK("J"),
    QUEEN("Q"),
    KING("K"),
    ACE("A");

    private final String label;

    Rank(String label) {
        this.label = label;
    }
```

Rank.java（続き）

```
    /** 自分が other より強ければ true */
    public boolean isStrongerThan(Rank other) {
        return compareTo(other) > 0;
    }

    public String label() {
        return label;
    }
}
```

Suit.java

```
/** カードのスート（マーク）。強さには関係しない。 */
public enum Suit {
    SPADES("Spades"),
    HEARTS("Hearts"),
    DIAMONDS("Diamonds"),
    CLUBS("Clubs");

    private final String label;

    Suit(String label) {
        this.label = label;
    }

    public String label() {
```

解答例 ・ 課題4（2/4）

クラスに切り出した例

Suit.java（続き）

```
        return label;
    }
}
```

Card.java

```
/** ランクとスートの組。どちらも欠けてはいけない。 */
public record Card(Rank rank, Suit suit) {
    public Card {
        Objects.requireNonNull(rank);
        Objects.requireNonNull(suit);
    }

    public boolean isStrongerThan(Card other) {
        return rank.isStrongerThan(other.rank);
    }

    public boolean isSameStrength(Card other) {
        return rank == other.rank;
    }

    public String label() {
        return rank.label() + " of " + suit.label();
    }
}
```

Deck.java

```
/** 山札。同じカードは入らず、空の山札からは引けない。 */
public class Deck {
    private final List<Card> cards;

    private Deck(List<Card> cards) {
        this.cards = cards;
    }

    /** 52 枚をそろえ、よく切った山札を作る */
    public static Deck shuffled(Random random) {
        List<Card> cards = new ArrayList<>();
        for (Suit suit : Suit.values()) {
            for (Rank rank : Rank.values()) {
                cards.add(new Card(rank, suit));
            }
        }
        Collections.shuffle(cards, random);
        return new Deck(cards);
    }

    public Card draw() {
        if (cards.isEmpty()) {
            throw new IllegalStateException("Deck is empty");
        }
        return cards.remove(0);
```

解答例 ・ 課題4（3/4）

クラスに切り出した例

Deck.java（続き）

```
    }

    public boolean isEmpty() {
        return cards.isEmpty();
    }

    public int size() {
        return cards.size();
    }
}
```

HighAndLowApp.java

```
public class HighAndLowApp {
    public static void main(String[] args) {
        long seed = args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
        Deck deck = Deck.shuffled(new Random(seed));
        Scanner scanner = new Scanner(System.in);

        Card current = deck.draw();
        System.out.println("First card: " + current.label());

        int streak = 0;
        boolean missed = false;
        while (!deck.isEmpty() && !missed) {
```

HighAndLowApp.java（続き）

```
            boolean high = askHigh(scanner, current);
            Card next = deck.draw();
            String result = judge(current, next, high);
            if (result.equals("Hit!")) {
                streak++;
            } else if (result.equals("Miss...")) {
                missed = true;
            }
            System.out.println("  Next card: " + next.label()
                    + " ... " + result + " (streak: " + streak
                    + ", " + deck.size() + " cards left)");
            current = next;
        }
        if (!missed) {
            System.out.println("No cards left.");
        }
        System.out.println();
        System.out.println("Game over. Your streak: " + streak);
    }

    static boolean askHigh(Scanner scanner, Card current) {
        String input = "";
        while (!input.equals("h") && !input.equals("l")) {
            System.out.print("Current card: " + current.label()
                    + ". Is the next card [h]igh or [l]ow? > ");
```

解答例 ・ 課題4（4/4）

クラスに切り出した例

HighAndLowApp.java（続き）

```
            input = scanner.nextLine().trim();
        }
        return input.equals("h");
    }

    static String judge(Card current, Card next, boolean high) {
        if (next.isSameStrength(current)) {
            return "Draw";
        }
        if (next.isStrongerThan(current) == high) {
            return "Hit!";
        }
        return "Miss...";
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

HighAndLowApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変えてみる

早く終わった人向け

演習 ・ 課題5

役割で分ける

judge() は、High か Low か（boolean high）と、結果を表す文字列で書き分けています。違いをそれぞれに任せ、ゲームの進行もクラスにします。

1. 予想（High・Low）を表すものを作り、当たりになる条件を予想ごとに持たせる
2. 結果（Hit・Draw・Miss）を表すものを作り、加える点と続けられるかを結果ごとに持たせる
3. 予想するプレイヤーと、ゲームを進めるクラスを作る
4. main は、部品をそろえて始めるだけにする

条件：ゲームを進めるクラスは、予想が High か Low か、結果がどれかを if で書き分けない

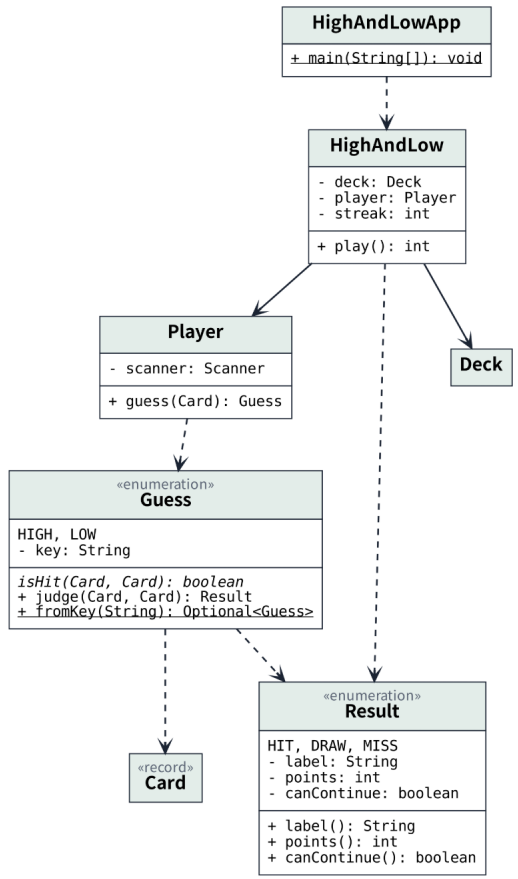
検討例 ・ 課題5

役割で分ける

役割	作るもの
2種類の予想と、入力の文字	enum Guess：HIGH, LOW, static fromKey()
当たりになる条件（予想ごとに違う）	Guess：isHit(Card, Card): boolean
同じ強さなら Draw、当たりなら Hit、それ以外は Miss	Guess：judge(Card, Card): Result
3種類の結果と、その表示	enum Result：HIT, DRAW, MISS, label()
加える点と、続けられるか	Result：points(), canContinue()
次のカードを予想する	Player：guess(Card): Guess
外れるか山札がなくなるまで繰り返す	HighAndLow：play(): int

検討例 ・ 課題5 クラス図

違いは、予想と結果に任せる



«enumeration» Guess

当たりになる条件 isHit() を、定数ごとに持つ

«enumeration» Result

加える点と、続けられるかを持つ

HighAndLow

High か Low か、結果がどれかを知らずに進める

Deck・Card

中身は課題4と同じ（図では省略）

解答例 ・ 課題5（1/3）

役割で分けた例

Guess.java

```
/** 予想。当たりになる条件を、予想自身が知っている。 */
public enum Guess {
    HIGH("h") {
        @Override
        boolean isHit(Card current, Card next) {
            return next.isStrongerThan(current);
        }
    },
    LOW("l") {
        @Override
        boolean isHit(Card current, Card next) {
            return current.isStrongerThan(next);
        }
    };

    private final String key;

    Guess(String key) {
        this.key = key;
    }

    abstract boolean isHit(Card current, Card next);

    /** 同じ強さなら DRAW、当たりなら HIT、それ以外は MISS */
    public Result judge(Card current, Card next) {
```

Guess.java（続き）

```
        if (next.isSameStrength(current)) {
            return Result.DRAW;
        }
        return isHit(current, next) ? Result.HIT : Result.MISS;
    }

    public static Optional<Guess> fromKey(String key) {
        for (Guess guess : values()) {
            if (guess.key.equals(key)) {
                return Optional.of(guess);
            }
        }
        return Optional.empty();
    }
}
```

解答例 ・ 課題5（2/3）

役割で分けた例

Result.java

```
/** 1 回の予想の結果。加える点と、続けられるかを知っている。 */
public enum Result {
    HIT("Hit!", 1, true),
    DRAW("Draw", 0, true),
    MISS("Miss...", 0, false);

    private final String label;
    private final int points;
    private final boolean canContinue;

    Result(String label, int points, boolean canContinue) {
        this.label = label;
        this.points = points;
        this.canContinue = canContinue;
    }

    public String label() {
        return label;
    }

    public int points() {
        return points;
    }

    public boolean canContinue() {
```

Result.java（続き）

```
        return canContinue;
    }
}
```

Player.java

```
/** 遊ぶ人。次のカードを予想する。 */
public class Player {
    private final Scanner scanner;

    public Player(Scanner scanner) {
        this.scanner = scanner;
    }

    public Guess guess(Card current) {
        while (true) {
            System.out.print("Current card: " + current.label()
                    + ". Is the next card [h]igh or [l]ow? > ");
            Optional<Guess> guess =
                    Guess.fromKey(scanner.nextLine().trim());
            if (guess.isPresent()) {
                return guess.get();
            }
        }
    }
}
```

解答例 ・ 課題5（3/3）

役割で分けた例

HighAndLow.java

```
/** 外れるか山札がなくなるまで、予想を繰り返す */
public class HighAndLow {
    private final Deck deck;
    private final Player player;
    private int streak = 0;

    public HighAndLow(Deck deck, Player player) {
        this.deck = deck;
        this.player = player;
    }

    /** 遊び終えたら、連続正解数を返す */
    public int play() {
        Card current = deck.draw();
        System.out.println("First card: " + current.label());

        Result result = Result.DRAW;
        while (!deck.isEmpty() && result.canContinue()) {
            Guess guess = player.guess(current);
            Card next = deck.draw();
            result = guess.judge(current, next);
            streak += result.points();
            System.out.println("  Next card: " + next.label()
                    + " ... " + result.label() + " (streak: "
                    + streak + ", " + deck.size() + " cards left)");
            current = next;
        }
        if (result.canContinue()) {
            System.out.println("No cards left.");
        }
        return streak;
    }
}
```

HighAndLow.java（続き）

HighAndLowApp.java

```
public class HighAndLowApp {
    public static void main(String[] args) {
        long seed = args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
        Deck deck = Deck.shuffled(new Random(seed));
        Player player = new Player(new Scanner(System.in));

        int streak = new HighAndLow(deck, player).play();
        System.out.println();
        System.out.println("Game over. Your streak: " + streak);
    }
}
```

課題

実装で意識すること

プラクティス	このゲームでの例
責務の分割 1つのクラスは1つのことを知る	強さの比べ方は Card と Rank、当たりの条件は Guess 数えるか・続けるかは Result、進める順番は HighAndLow が知っている
カプセル化 不変条件を守る	カードは、ランクとスートが必ずそろっている 山札には同じカードが入らず、空の山札からはめくれない
多態性 種類を知らずに任せる	HighAndLow は High と Low を区別しない 当たりの条件は予想が、続けられるかは結果が知っている

データとそれを扱う処理は一緒に置き、種類ごとの違いはそれぞれに任せる。

課題 ・ ふりかえり

最初の main と比べてみよう

課題2と課題5のコードを並べて、次のことを話し合いましょう。

1. 「同じ強さ」を予想できるようにするとき、それぞれどこを直すか

2. スートにも強さをつけるとき、それぞれどこを直すか

3. 課題4・5で分けた境目は、課題1で見つけた「もの」と同じだったか

分けた境目が、変更とテストのしやすさにつながっている。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

HighAndLowApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変えてみる

早く終わった人向け

演習 ・ 発展課題

仕様を変えてみる

早く終わった人は、次の変更に挑戦しましょう。変更したら、直したクラスとメソッドの数を数えてください。

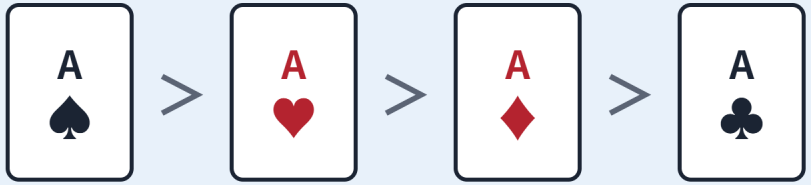
1. 「同じ強さ」を予想できるようにする（[s]ame：同じ強さなら当たり）
2. 何回か続けて遊べるようにし、最高記録を表示する
3. ジョーカーを1枚加える（どのカードと比べても Draw になる）

直す箇所が少なく、1か所に集まっているほど、変更に強い設計です。HighAndLow を直す必要があったかどうかにも注目しましょう。

演習 ・ 付録課題

スートにも強さをつける

ランクが同じときは、スートで強さを決めることにします。強さの順は、スペード ＞ ハート ＞ ダイヤ ＞ クラブ とします。

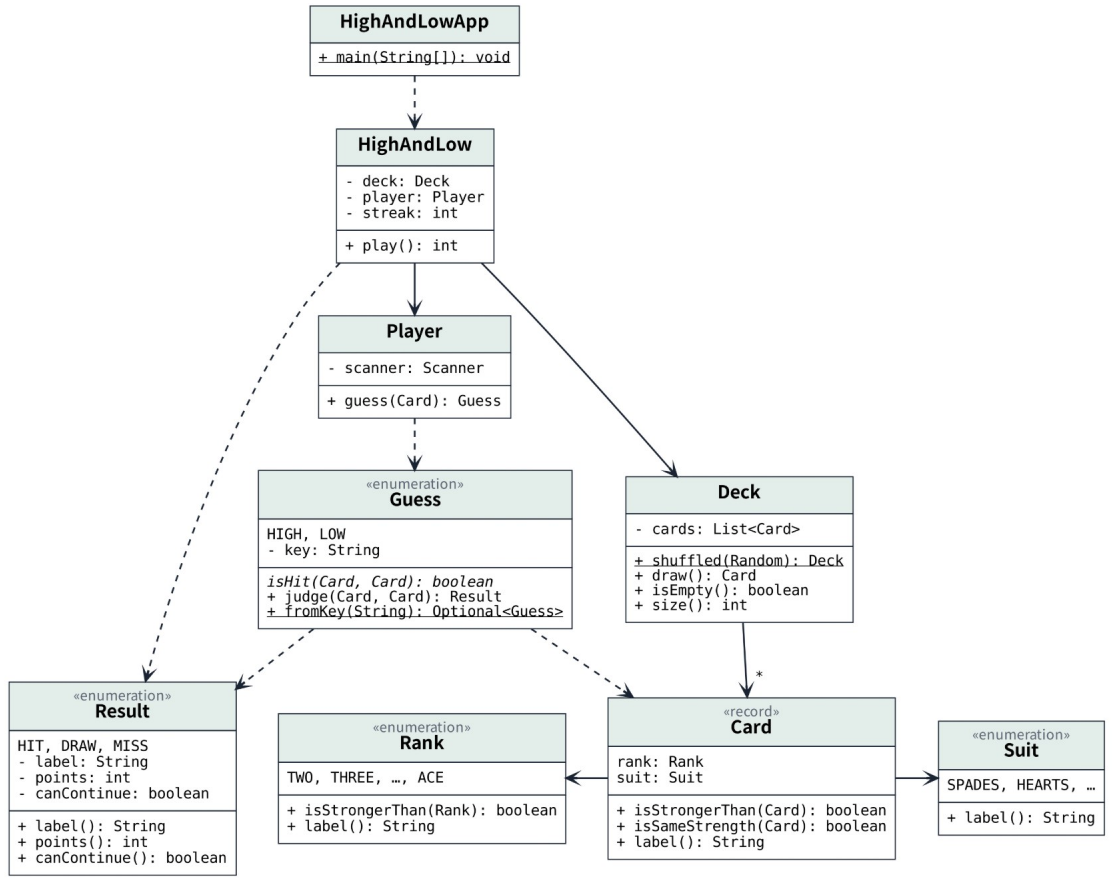


1. この変更で、「同じ強さ」のカードはどうなるか

2. どのクラスのどのメソッドを直せばよいか。直さなくてよいクラスはどれか

検討例 ・ 付録 クラス図

全体のつながり



HighAndLowApp・HighAndLow

部品をそろえ、外れるか山札がなくなるまで進める

Player・Guess・Result

予想と結果ごとの違いを、それぞれに任せる（課題5）

Deck・Card

データと処理をまとめて閉じ込める（課題4）

Rank・Suit

強さの順を、enum の並び順で表す